Playing Atari with Deep Reinforcement Learning

날짜	@2026년 3월 30일
다중 선택	

Paper Review

Playing Atari with Deep Reinforcement Learning

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural...

https://arxiv.org/abs/1312.5602

Keyword : Reinforcement Learning with Deep Neural Network , Raw Visual Input, Experience Replay

1. Introduction

논문이 다루는 분야

Deep Reinforcement Learning

고차원 sensory input으로부터 agent의 제어 정책을 직접 학습하는 딥러닝 기반 강화학습

해당 task에서 기존 연구 한계점

- **기존 강화학습의 한계**
  - 과거 강화학습은 사람이 직접 설계한 feature에 의존하여 시스템 성능이 feature 표현의 품질에 의존함
- **최근 딥러닝의 발전**
  - 다양한 신경망 구조(CNN, RNN, MLP)와 지도/비지도 학습을 이용해 raw sensory data에서 고수준 특징을 추출하는 것이 가능해지면서 컴퓨터 비전과 언어 인식 분야의 발전을 이끔

    → Sensory data를 사용하는 강화학습에도 비슷한 기법이 효과적일 것으로 예상
- **강화학습에 딥러닝 적용 시 발생하는 난제**

	딥러닝	강화학습
Sparse & delayed reward	• 대규모의 학습 데이터가 필요 • Input과 target을 직접 연결하여 학습	• Sparse, noisy 보상으로 학습 • 행동과 보상 간에 수천 timestep의 delay가 발생
Highly correlated data	대부분의 알고리즘이 데이터 샘플의 독립성을 가정	데이터 간 상관관계가 높은 sequence를 다룸
Non-stationary distribution	고정된 데이터 분포를 가정	알고리즘이 새로운 행동을 학습함에 따라 데이터 분포가 변화함

논문의 contributions

- **DQN의 Contributions**

  "**CNN**을 이용해 raw video data로부터 제어 정책을 성공적으로 학습할 수 있음을 증명"
  - Q-learning 변형 알고리즘과 SGD로 가중치 업데이트

- Experience replay mechanism을 이용해 이전의 상태 변이를 랜덤 샘플링하여 학습함으로써 데이터 상관관계 및 분포 변동성 문제를 완화

- **성과**

  Test 환경)

  - 고차원의 visual input을 가진 다양한 Atari 2600 게임에 적용
  - 게임별 정보나 사람이 설계한 visual feature 없이 오로지 video input으로 학습
  - 네트워크 구조와 하이퍼파라미터도 모든 게임에서 동일하게 사용

  Result)

  - 7개 중 6개의 게임에서 이전 강화학습 알고리즘을 뛰어넘음
  - 그 중 3개의 게임에서 숙련된 인간 플레이어의 점수를 추월함

2. Related Work

- **강화학습 성공 사례 - TD-gammon**

  강화학습과 self-play를 통해 인간 이상의 수준에 도달한 주사위 놀이 프로그램

  - **특징** : Q-러닝과 유사한 model-free 알고리즘을 사용했으며, 하나의 은닉층을 가진 MLP로 가치 함수를 근사
  - **한계** : 체스, 바둑, 체커 등 다른 게임에 적용했을 때는 실패함. TD-Gammon의 성공은 주사위의 무작위성이 상태 공간 탐색을 돕고 가치 함수를 부드럽게 만든 특수한 경우라는 인식이 확산됨.

- **강화학습과 딥러닝 결합의 문제**

  - **Divergence** : Q-learning과 같은 model-free 알고리즘을 non-linear function approximator나 off-policy 학습과 결합할 경우, Q-network가 **발산**할 수 있음이 증명됨

    → 대부분의 강화학습 연구는 수렴이 보장되는 linear function approximator에 집중됨.

- **최근 강화학습과 딥러닝 결합 연구**

  - **DNN을 이용한 환경 $\mathcal{E}$ 추정** : 제한된 Boltzmann machine을 이용해 가치 함수나 정책 추정
  - **일부 수렴성 확보** : Gradient temporal-difference 방법으로 **non-linear** function approximator & **fixed** policy 평가나 **linear** function approximator & **control** policy 학습에서 일부 수렴성을 확보함

  → **Nonlinear control**까지 확장되지는 못함.

- **가장 유사한 연구** : **NFQ**(Neural fitted Q-learning)

  - RPROP 알고리즘으로 DQN과 유사한 손실 함수를 최적화
  - **차이점**

    1. **연산 비용** : NFQ는 batch update를 사용하여 연산 비용이 전체 데이터셋 크기에 비례하지만, DQN은 SGD를 사용하여 낮은 상수의 연산 비용을 가지며, 대규모 데이터셋으로 확장 가능함
    2. **학습 방식** : NFQ는 raw visual input을 사용해 간단한 현실 세계 task에 성공적으로 적용됐으나, auto-encoder로 저차원 표현을 먼저 학습한 후 NFQ을 적용하는 방식인 반면, DQN은 visual input으로부터 직접 학습하는 End-to-End 방식 → 행동-가치 구분과 직접 관련된 특징 학습 가능

  - 기타 관련 연구 : 과거에 Q-learning이 experience replay와 간단한 신경망과 결합된 적이 있으나 raw visual input이 아닌 저차원 상태에서 시작함

- **Atari 2600 emulator를 활용한 강화학습 연구**

  - Linear function approximator와 일반적인 visual feature을 사용하는 표준 강화학습 알고리즘을 적용한 연구에서 Atari를 도입

- HyperNEAT 연구에서 진화한 구조를 적용하여 각 게임에 맞는 전략을 학습한 신경망을 제시했으나, Atari 게임의 설계 결함을 이용한 것에 그침

3. 제안 방법론

Main Idea

- **목표** : **강화학습** 알고리즘을 1) RGB 이미지에서 바로 작동하며, 2) SGD로 효율적으로 처리하는 **DNN**과 연결하는 것
- **참고 구조** : TD-Gammon 구조
  - On-policy 방식으로 알고리즘이 환경과 상호작용하며, 가치 함수를 추정하는 신경망의 파라미터를 업데이트
- **DQN (Deep Q-learning) 알고리즘**

  Methods)

  - **Experience replay**

    Agent의 매 time-step의 경험을 replay memory에 저장하고, Q-learning 시에는 메모리에서 랜덤하게 경험을 샘플링하여 minibatch 업데이트

$$
\begin{gathered}
Episode: \ e_t = (s_t, a_t, r_t, s_{t+1}) \\
Dataset: \ D = e_1, ..., e_N \\
e \sim D
\end{gathered}
$$

    → 반드시 off-policy을 사용

  - ϵ**-greedy policy**
  - **Fixed length representation :** 신경망에 임의의 길이를 가진 시퀀스를 입력으로 넣기는 어려우므로 함수 ϕ로 생성된 고정된 크기의 표현을 사용

Advantages) Standard Online Q-learning과 비교

1. **데이터 효율성** : 각 step의 경험이 여러 번의 가중치 업데이트에 사용될 수 있음
2. **상관관계 제거** : 연속적인 샘플들은 강한 상관관계가 있어 연이은 학습은 비효율적임. 랜덤 샘플링을 통해 업데이트의 분산을 감소시킴
3. **학습 안정성** : On-policy로 학습하면, 현재 파라미터가 다음에 학습될 데이터 샘플을 결정하면서 데이터 분포가 치우치게 됨. 원치 않는 feedback loop에 빠지거나 파라미터가 local minimum에 빠지거나 발산하기 쉬움. → Experience replay를 통해 행동 분포가 여러 상태를 평균낸 형태가 되면서 학습이 부드러워지고 파라미터의 진동이나 발산을 억제함

In practice)

- **데이터 저장 :** 가장 최근 N개의 경험만 저장
- **샘플링 방식 :** 업데이트 시 데이터셋D에서 균등한 확률로 샘플을 랜덤 추출

한계

- Overwrite : 메모리 용량에 한계가 있어 메모리 버퍼가 중요도를 구분하지 않고 오래된 경험을 최신 경험으로 덮어씀
- Uniform importance : 균등 샘플링은 모든 경험에 동일한 중요도를 부여함

개선 방향

- Sophisticated sampling : 가장 많이 배울 수 있는 경험을 강조하는 정교한 샘플링 전략

4.1 Preprocessing and Model Archcitecture

Preprocessing)

1. **Down-sampling** : RGB를 gray-scale로 변환 후 110 x 84로 down-sampling
    - Raw Atari frame은 210 x 160 크기에 128개의 컬러를 가지고 있어서, 연산 비용을 줄이기 위해 입력 차원을 줄임
2. **Image crop** : 84 × 84 크기로 크롭한 이미지를 최종 입력으로 사용
    - AlexNet에서 2D Conv 연산을 위해 GPU를 최적화했던 방식을 사용하는데, 이때 정사각 입력이 필요함
3. **Frame stacking** : 시퀀스의 마지막 4개 프레임을 전처리 후 쌓아서 Q-function의 입력을 만듦

Model architecture)

- **기존 방식** : 상태와 행동을 입력으로 넣어 Q-value를 출력
    - **단점** : 행동 개수만큼 Q-value 계산을 위한 forward pass가 필요하다는 문제가 있음
- **DQN 방식** : 상태를 입력으로 넣어 가능한 행동별 Q-value를 출력
    - **장점** : 하나의 forward pass만으로 주어진 상태에서 가능한 모든 행동에 대한 Q-value 계산이 가능함
- **DQN 모델 구조**
    1. **Input :** 84 × 84 × 4 이미지
    2. **Conv 1 :** 16개의 8 x 8 filter (stride 4) + ReLU
    3. **Conv 2 :** 32개의 4x4 filter (stride 2) + ReLU
    4. **FC Layer :** 256개의 유닛을 가진 Fully-connected layer + ReLU
    5. **Output Layer:** 가능한 행동의 개수만큼의 유닛을 가진 Fully-connected layer (4~18개)

4. 실험 및 결과

Dataset & Setting

- **실험 대상** : 7개의 ATARI 게임→ Beam Rider, Breakout, Enduro, Pong, Q*bert, Seaquest, Space Invaders
- **실험 환경**
    - **모델 설정** : 각 게임에 특화된 정보 없이도 다양한 게임에 강건함을 보여주기 위해 모든 게임에 동일한 네트워크 구조, 알고리즘, 하이퍼파라미터를 사용
    - **보상 구조** : 게임마다 점수 범위가 매우 다르기 때문에, 양수 보상은 1, 음수 보상은 -1, 0은 그대로 0으로 고정
        - **장점** : 이러한 reward clipping은 오차 미분 값의 범위를 제한하고 여러 게임에서 동일한 학습률을 사용하기 쉽도록 함
        - **단점** : 보상의 크기 차이를 구분하지 못해 agent 성능에 영향을 줄 수 있음
    - **하이퍼파라미터**
        - Optimizer : RMSProp
        - Batch size : 32
        - ϵ : 초기 100만 프레임동안 1에서 0.1까지 선형적으로 감소시킨 후 0.1로 고정
    - **Frame-skipping 기법** : agent가 각 프레임마다가 아니라 k개의 프레임마다 보고 행동을 선택하며, 건너뛴 프레임에서는 가장 마지막 행동을 반복
        - **장점** : 한 스텝을 진행하는 데 드는 연산량을 줄이기 때문에, runtime 증가 없이 k배 더 많은 게임 플레이 가능

- **k 설정** : k=4를 기본으로 하되, Space Invaders에서는 레이저가 깜빡이는 주기 때문에 k=3으로 설정하여 레이저가 보이도록 함

결과

5.1 Training and Stability

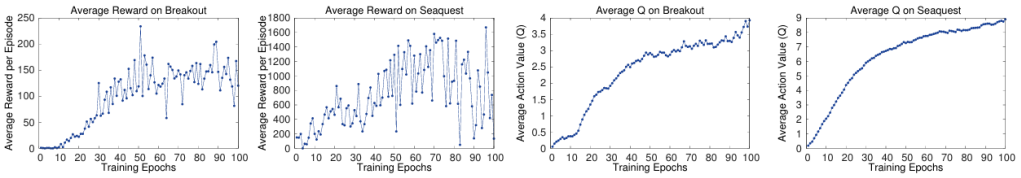


Figure 2 : Average total reward(left), Average predicted Q(right)

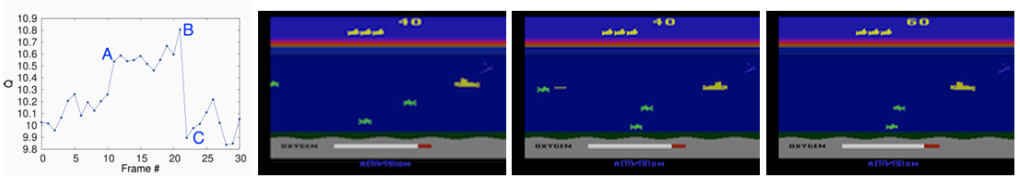
성능 평가 지표)

- **Average total reward** : 에피소드나 게임 내에서 agent가 수집한 총 보상을 게임 수로 나눈 값
  - **단점** : 정책 가중치의 작은 변화가 정책이 방문한 상태의 분포를 크게 바꿔 매우 노이즈가 심함
- **Average predicted Q :** 에이전트가 특정 상태에서 얻을 수 있는 할인 보상의 추정치
  - **계산 방식** : 학습 시작 전에 랜덤 정책으로 상태 세트를 수집하여 추적 대상으로 고정해놓고, 각 상태에서 추정된 최대 Q의 평균을 추적
  - **장점** : Average total reward 보다 훨씬 매끄러우며, 점차 상승하며 수렴하고 있음을 보임

발산 방지 : DQN 모델에서는 발산 문제가 발생하지 않음
→ 이론적으로 수렴이 보장되지 않지만, 강화학습 signal과 SGD를 사용해 거대한 신경망을 안정적으로 학습시킬 수 있음

5.2 Visualizing the Value Function

학습된 가치 함수를 시각화하여 분석



예시) Seaquest 게임

1. Point A : 화면 왼쪽에 적이 등장하자 예측 가치 Q가 급격히 상승함
2. Point B : 에이전트가 적을 향해 어뢰를 발사하고, 어뢰가 적에게 명중하기 직전에 Q값이 정점을 찍음
3. Point C : 적이 화면에서 사라진 후, Q값은 다시 원래의 수준으로 떨어짐

→ DQN이 상당히 복잡한 일련의 사건에 따라 가치 함수가 어떻게 발전해야 하는지를 학습할 수 있음을 입증

5.3 Main Evaluation

강화학습 분야에서 가장 좋은 성능을 보인 알고리즘들 및 인간 플레이어와 비교

- **비교 대상**

- **Sarsa/Contingency**
  - Sarsa : Atari 태스크에 맞춰 사람이 직접 설계한 feature set을 기반으로 선형 정책을 학습
  - Contingency : Sarsa와 기본적 접근은 비슷하나, agent의 제어 하에 있는 화면 부분의 학습된 표현을 feature set에 추가

  → 두 방법 모두 배경 제거와 128개의 색상을 분리된 채널로 처리하는 방식을 사용하여 상당한 사전 지식을 포함함

  ↔ **DQN**은 raw RGB 스크린샷을 그대로 입력으로 받고 객체 탐지를 스스로 학습
- **랜덤 정책** : 균일 확률 분포에서 행동을 선택
- **HNeat :** 진화 정책 탐색 방식
  - **Best :** Atari 화면에서 객체의 위치와 유형을 출력하는 객체 탐지 알고리즘으로 얻은 결과를 반영
  - **Pixel** : 각 채널의 객체 라벨 맵을 나타내는 8개의 색상 채널 표현을 사용

  → 결정론적 시퀀스를 찾는 데 의존하여 랜덤 변형에 대한 일반화가 어려움. 따라서 알고리즘을 최고 점수를 받은 에피소드 하나로만 평가함.

  ↔ **DQN**은 ϵ-greedy 전략에 따른 시퀀스로 평가되므로 다양한 상황에 일반화 가능
- **숙련된 인간 플레이어** : 약 2시간 동안 각 게임을 플레이한 후 얻은 점수의 중앙값으로 평가

- **결과**
  - **알고리즘과 비교** : Space Invaders제외 모든 게임에서 최댓값으로 평가한 결과(DQN Best)뿐만 아니라 평균낸 결과(DQN)도 기존 방법들보다 좋은 성능을 보임
  - **인간 플레이어와 비교**
    - Breakout, Enduro, Pong : DQN이 더 좋은 점수
    - Beam Rider : 근접한 점수
    - Q*bert, Seaquest, Space Invaders : DQN이 훨씬 낮은 점수 → 긴 시간에 걸친 전략을 필요로 하기 때문에 난도가 더 높음

5. 결론

- **강화학습에 새로운 딥러닝 모델 제시 :** Raw pixels만을 입력으로 사용하여 Atari 2600 컴퓨터 게임의 어려운 제어 정책을 마스터함
- **Q-learning의 변형 :** SGD와 Experience replay memory를 결합하여 강화학습을 위한 DNN 학습의 난도를 완화
- **SOTA 달성** : 게임마다 모델 구조나 하이퍼파라미터를 조정하지 않고도 테스트한 7개 게임 중 6개에서 SOTA 달성

관련 개념 정리

교재 : 파이썬과 케라스로 배우는 강화학습
범위 : 1장 강화학습 개요 (p. 4-18), 2장 강화학습 기초1: MDP와 벨만 방정식 (p. 28-60), 3장 강화학습 기초2: 그리드월드와 다이내믹 프로그래밍 (p. 67-75, 92-96)

강화학습 개요

강화학습의 개념

스키너의 강화 연구

행동심리학에서 "강화(Reinforcement)"는 동물이 시행착오를 통해 학습하는 방법으로, 행동심리학자 스키너(Skinner)가 처음 제시한 개념이다.

스키너의 쥐 실험

- **실험 방법** : 굶긴 쥐를 상자에 넣고, 쥐가 우연히 지렛대를 누르면 먹이가 나오도록 한다.
- **실험 결과** : 쥐는 지렛대를 누르는 행동과 먹이의 상관관계를 모르는 채 다시 돌아다니지만, 우연히 다시 지렛대를 누르면서 행동과 먹이의 관계를 알게 되고 점점 지렛대를 자주 누르게 된다.

→ '**강화**'는 이전에 배우지 않았던 것을 직접 시도하면서 행동과 좋은 보상 사이의 상관관계를 학습하고 좋은 보상을 얻게 해주는 행동을 점점 더 많이 하는 것

우리 주변에서의 강화

처음 걸음을 배울 때와 같이 사람도 어떤 것을 처음 배울 때 사전지식이 없이 행동과 보상 사이의 연관관계를 학습하게 된다. 사람과 동물에게 학습의 기본이 되는 **강화**라는 개념이 **강화학습의 모티프**가 된 것이다.

머신러닝과 강화학습

머신러닝은 인공지능의 한 범주로서 컴퓨터가 스스로 학습하게 하는 알고리즘을 개발하는 분야로, 미리 프로그래밍된 대로 작동하는 것이 아니고 주어진 데이터를 토대로 스스로 성능을 높여가는 것을 말한다.

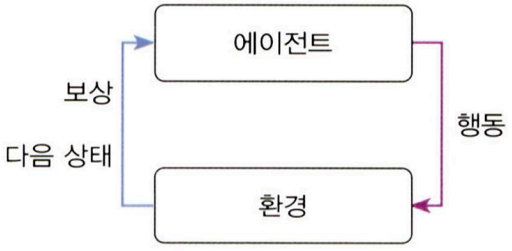
- **지도학습(Supervised Learning)** : "정답"을 알고 있는 데이터를 이용해 컴퓨터는 자신이 낸 답과 정답의 차이를 통해 지속해서 학습.
  - **예시** : 회귀분석(Regression)과 분류(Classification) 등
- **비지도학습(Unsupervised Learning)** : 지도학습과는 다르게 정답이 있는 것이 아니라, 여러 개의 점이 주어지면 비슷한 것끼리 묶어주는 식의 학습
  - **예시** : 군집화(Clustering)
- **강화학습** : 정답이 주어지는 것이 아니고 주어진 데이터에 대해 학습하는 것도 아니며, 보상을 통해 학습

지도학습 vs 강화학습

- **지도학습** : 직접적인 정답을 통해 **오차**를 계산해서 학습
- **강화학습** : 자신의 행동의 결과로 나타나는 **보상**을 통해 학습
  - 보상은 컴퓨터가 선택한 행동에 대한 환경의 반응으로, 직접적인 답은 아니지만 간접적인 정답의 역할

스스로 학습하는 컴퓨터, 에이전트

강화학습을 통해 스스로 학습하는 컴퓨터 → **에이전트(Agent)**



- **상호작용 과정**
  1. 환경에 대해 사전지식이 없는 상태에서 학습
  2. 자신이 놓인 환경에서 자신의 **상태**를 인식한 후 **행동**
  3. 환경이 에이전트에게 **보상**을 주고 다음 상태를 알려줌

  → 자신의 행동과 행동에 따른 보상을 통해 어떤 행동을 해야 좋은 결과를 얻게 되는지 학습
- **강화학습**
  - **목적 :** 에이전트가 환경을 탐색하면서 얻는 보상들의 합을 최대화하는 최적의 행동양식, 또는 **정책**을 학습
  - **장점** : 환경에 대한 정확한 지식이 없어도 시행착오를 통해 학습 가능

  보상 설계의 중요성
  보상은 양수(=상) 또는 음수(=벌)로 설정 가능하다. 상벌을 적절히 융합하면 에이전트가 자신이 무엇을 해야 하는지 더 명확하게 알 수 있어 효과적인 학습이 가능하다.

강화학습 문제

강화학습은 결정을 **순차적**으로 내려야 하는 문제에 적용된다. 다이나믹 프로그래밍(Dynamic Programming)이나 진화 알고리즘(Evolutionary Algorithm)도 순차적 행동 결정 문제를 푸는 데 적용할 수 있으나, 각기 한계가 있으며 강화학습이 그 한계를 극복할 수 있다.

순차적 행동 결정 문제

순차적으로 행동을 결정해야 하는 문제를 수학적으로 정의하는 방법 → **MDP(Markov Decision Process)**

순차적 행동 결정 문제의 구성 요소

MDP 구성 요소)

- **상태(State)** : 에이전트가 상황을 판단하고 행동을 결정하기 위한 정보로 위치 같은 정적 요소뿐 아니라 속도 같은 동적 요소도 포함 (엄밀히는 "관찰"이 더 정확한 표현)
- **행동(Action)** : 에이전트가 특정 상태에서 취할 수 있는 행동(ex- 상, 하, 좌, 우)으로 처음에는 무작위로 행동하다가 학습하면서 특정 행동의 확률을 높임
- **보상(Reward)** : 에이전트가 어떤 행동이 좋은 행동인지를 학습할 수 있는 유일한 정보. 에이전트에 속하지 않은 환경의 일부이며, 어떤 상황에서 얼마의 보상이 나오는지 미리 알 수 없음.
- **정책(Policy)** : 모든 상태에 대해 어떤 행동을 해야 하는지 정해놓은 것으로 순차적 행동 결정 문제에서 구해야 할 답. 보상의 합이 최대가 되는 가장 좋은 정책을 최적 정책(Optimal Policy)이라 함.

방대한 상태를 가진 문제에서의 강화학습

강화학습은 최근에 방대한 상태를 가진 문제에서 뛰어난 성능을 보여주고 있다.

- **방대한 상태를 가진 문제**
  - **바둑** : 가능한 경우의 수는 10^{360}으로 우주의 원자 수인 10^{80}에 비해서도 월등히 많은 수
  - **로봇** : 관찰하는 정보와 행동, 보상이 모두 연속적이기 때문에 사실상 경우의 수가 무한대.

  → 방대한 상태에 강화학습을 적용하려면 **인공신경망**을 사용하여 상태 정보를 함수로 근사해야 함

강화학습 기초1: MDP와 벨만 방정식

MDP

MDP는 순차적으로 행동을 결정해야 하는 문제를 수학적으로 정의한 것으로, **상태, 행동, 보상함수, 상태 변환 확률, 할인율**로 구성된다.

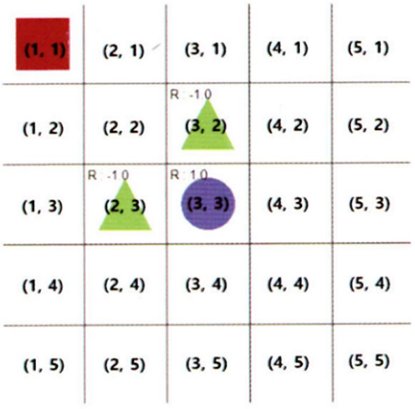
- **그리드월드(Grid World)** : MDP를 이해하기 위해 격자로 이뤄진 환경에서 문제를 푸는 각종 예제

상태

- S = 에이전트가 관찰 가능한 상태의 집합
- S_t = 시간 t일 때의 상태로, 어떤 상태가 될지 정해지지 않은 확률변수

> "시간 t에서의 상태 S_t가 어떤 상태 s다"
> → $S_t = s$

그리드월드)



- **상태** = 격자 위의 각 **좌표**

> **5×5 그리드 → 25개 상태**
> $S = \{(1,1),(1,2),(1,3),\ldots,(5,5)\}$

행동

- A = 에이전트가 상태 S_t에서 가능한 행동의 집합
  - 보통 에이전트가 할 수 있는 행동은 모든 상태에서 같음
- A_t = t라는 시간에 집합 A에서 선택한 행동으로, 어떤 행동을 할지 정해지지 않은 확률변수

> "시간 t에 에이전트가 특정한 행동 a를 하는 경우"
> → $A_t = a$

- 바람 같은 예상치 못한 요소가 있으면 의도한 상태에 도달하지 못할 수 있음 → 상태 변환 확률로 표현

그리드월드)

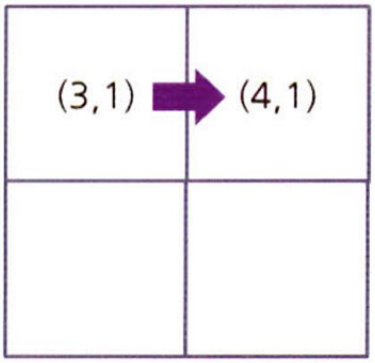


그림 2.4

- **행동**

> $A = \{up, down, left, right\}$
> **그림 2.4)** 시간 t에서 상태가 (3,1)이고, $A_t = right$라면 다음 시간의 상태는 (4, 1)

보상함수

보상은 에이전트가 학습할 수 있는 유일한 정보로서 환경이 에이전트에게 주는 정보이다.

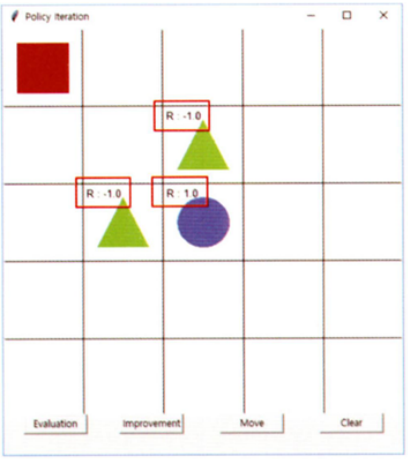
- **보상함수** : 상태 s에서 행동 a를 했을 경우에 받을 보상에 대한 **기댓값**(Expectation)

$$r(s, a) = E[R_{t+1}|S_t = s, A_t = a]$$

  - 기댓값은 어떤 정확한 값이 아니라 나오게 될 숫자에 대한 예상을 의미함. 환경에 따라 에이전트가 같은 상태에서 같은 행동을 취해도 다른 보상을 줄 수 있으므로 기댓값으로 표현하는 것.
  - $S_t = s, A_t = a$: 현재의 조건이자 입력
  - R_{t+1} : 에이전트가 받는 보상

    → 시간 t에서 행동을 했을 때, 환경이 에이전트에게 다음 상태와 보상을 알려주는 시점은 $t + 1$

그리드월드)



- 보상

> 파란색 동그라미가 있는 상태로 가는 행동 → +1
> 초록색 세모가 있는 상태로 가는 행동 → -1

상태 변환 확률

에이전트가 어떤 행동을 취하면 상태가 $s \rightarrow s'$으로 변하는데, 이 변화에는 확률적인 요인이 들어간다. 이를 수치적으로 표현한 것이 상태 변환 확률이다.

- **상태 변환 확률** : 상태 s에서 행동 a를 취했을 때 다른 상태 s'에 도달할 확률

$$P_{ss'}^{a} = P[S_{t+1} = s' | S_t = s, A_t = a]$$

  - 보상과 마찬가지로 에이전트가 알지 못하는 값으로, 환경의 일부 (= 환경의 모델)

할인율

에이전트는 항상 현재에 판단을 내리기 때문에 현재에 가까운 보상일수록 더 큰 가치를 가진다. 미래에 받을 보상을 현재의 가치로 환산하기 위해 할인이라는 개념을 도입한다.

- **할인율(Discount Factor)** : 시간에 따라 할인하는 비율

$$\gamma \in [0, 1]$$

  - 보상에 곱해지면 보상이 감소
- 현재의 시간 t로부터 시간 k만큼 지났을 때 받을 미래 보상의 현재 가치

$$\gamma^{k-1} R_{t+k}$$

  - 더 먼 미래에 받은 보상일수록 현재의 에이전트는 더 작은 값으로 받아들임

정책

정책은 모든 상태에서 에이전트가 할 행동을 정해놓은 것으로, 상태를 입력으로 받아 행동을 출력으로 내보내는 일종의 함수이다.

- **정책** : 시간 t에 상태 s에 있을 때 가능한 행동 중 a를 할 확률

$$\pi(a|s) = P[A_t = a | S_t = s]$$

  - 정책은 각 상태에서 하나의 행동만을 나타낼 수도 있고, 확률적으로 여러 행동을 나타낼 수도 있음 (ex - a_1 = 10%, a_2 = 90%)
- **최적 정책** : 수많은 정책 중 가장 많은 보상을 받게 하는 최고의 정책

MDP를 통한 강화학습 과정 요약

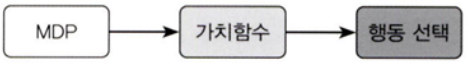
1. 에이전트가 현재 상태에서 앞으로 받을 보상들을 고려해서 행동을 결정
2. 환경은 에이전트에게 실제 보상과 다음 상태를 알려줌
3. 이를 반복하면서 앞으로 받을 것이라 예상했던 보상과 실제로 받은 보상을 토대로 가치함수와 정책을 바꿔나감
   - **가치함수(Value Function) :** 앞으로 받을 것이라 예상하는 보상 함수

→ 이러한 과정을 충분히 반복하여 가장 많은 보상을 받게 하는 정책을 학습

가치함수

가치함수

에이전트는 MDP를 통해 최적 정책을 찾아야 하는데, 어떤 행동을 하는 것이 좋은지를 판단하기 위해 앞으로 받을 보상을 표현한 함수가 **가치함수**이다.

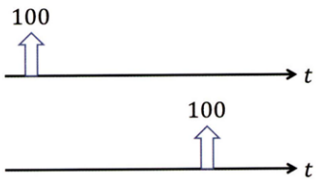


보상을 합하는 방법)

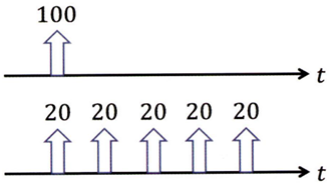
- **단순합 :** 현재 시간 t로부터 에이전트가 행동을 하면서 받는 보상들의 단순합

$$R_{t+1} + R_{t+2} + R_{t+3} + R_{t+4} + \dots$$

  - 단순합에는 세 가지 문제가 있음
    1. 현재 받는 보상이나 미래에 받는 보상이나 똑같이 취급됨



    2. 한 번에 큰 보상을 받는 것과 여러 번 나눠서 받는 것을 구분할 수 없음



    3. 시간이 무한대일 경우 시간마다 받는 보상 값이 달라도 보상의 합이 무한대가 되어 구분 불가

$$0.1 + 0.1 + 0.1 + \cdots = \infty$$
$$1 + 1 + 1 + \cdots = \infty$$

    → 시간 t에 있었던 상태의 가치를 전확히 판단하기 어려움

- **반환값 :** 미래 보상을 현재의 보상으로 변환하는 할인율을 적용한 보상들의 합

$$G = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots$$

  - 에이전트와 환경이 유한한 시간동안 상호작용하는 것을 **에피소드**라 부르며, 에피소드가 끝난 후 각 시점에서 그때 이후로 받은 보상으로 정산하여 반환값을 구함
    - **예시)** t=1 부터 t=5 까지 진행한 경우 반환값

$$
\begin{gathered}
\mathrm{G_1 = R_2 + \gamma R_3 + \gamma^2 R_4 + \gamma^3 R_5 + \gamma^4 R_6} \\
\mathrm{G_2 = R_3 + \gamma R_4 + \gamma^2 R_5 + \gamma^3 R_6} \\
\mathrm{G_3 = R_4 + \gamma R_5 + \gamma^2 R_6} \\
\mathrm{G_4 = R_5 + \gamma R_6} \\
\mathrm{G_5 = R_6}
\end{gathered}
$$

가치함수의 정의)

가치함수는 반환값에 대한 기댓값으로, 특정 상태의 가치를 판단할 수 있다.

$$v(s) = E[G_t | S_t = s]$$

- **입력** : 상태
- **출력** : 앞으로 받을 보상의 합
- 에이전트는 가치함수를 통해 갈 수 있는 상태들의 가치를 알고, 그중 가장 가치가 높은 상태를 선택
- 가치함수를 반환값으로 표현하면,

$$
\begin{gathered}
v(s) = E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \ldots | S_t = s] \\
\rightarrow v(s) = E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} \ldots) | S_t = s] \\
\rightarrow v(s) = E[R_{t+1} + \gamma G_{t+1} | S_t = s]
\end{gathered}
$$

  $R_{t+2} + \gamma R_{t+3} \ldots$부분은 실제로 받은 보상이 아니라 앞으로 받을 것이라 예상하는 보상이므로 가치함수로 표현하면,

$$v(s) = E[R_{t+1} + \gamma v(S_{t+1}) | S_t = s]$$

정책을 고려한 가치함수)

에이전트가 앞으로 받을 보상을 생각할 때 정책을 고려하지 않으면 안 된다. 상태에서 상태로 넘어갈 때 에이전트는 행동을 해야 하고, 각 상태에서 행동을 하는 것이 에이전트의 정책이기 때문이다.

- 정책을 포함해 가치함수를 표현하면,

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

  - 이는 현재 상태의 가치함수 $v_\pi(s)$와 다음 상태의 가치함수$v_\pi(S_{t+1})$ 사이의 관계를 말해주는 방정식 **벨만 기대 방정식**과 동일함

큐함수

- **상태 가치함수(State Value Function)** : 상태를 입력으로 받아 그 상태에서 앞으로 받을 보상의 합을 출력하는 함수
  - 에이전트는 어떤 상태에 있는 것이 좋은지를 알고 다음에 어떤 상태로 가야 할지 판단 가능

  → 하지만 다음 상태로 가기 전에 선택한 **행동**에 따라 즉각적으로 받는 보상이 달라질 수 있음. 상태 변환 확률에 따라 가고자 한 상태와 다른 상태로 가는 경우도 고려하여 행동을 선택해야 함.

- **행동 가치함수 = 큐함수(Q Function)** : 상태와 행동 두 가지를 입력으로 받아 어떤 상태에서 어떤 행동이 얼마나 좋은지 알려주는 함수

$$q_\pi(s, a)$$

  - 강화학습에서 에이전트가 행동을 선택하는 기준으로 보통 큐함수를 사용

- **큐함수의 벨만 기대 방정식**

$$q_\pi(s,a) = E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

상태 가치함수와 큐함수의 관계

$$v_\pi(s) = \Sigma\, \pi(a|s) q_\pi(s,a)$$

1. 각 행동을 했을 때 앞으로 받을 보상인 큐함수 $q_\pi(s,a)$를 $\pi(a|s)$에 곱함
2. 모든 행동에 대해 큐함수와$\pi(a|s)$를 곱한 값을 더하면 가치함수가 됨

벨만 방정식

벨만 기대 방정식

- **벨만 기대 방정식 :** 현재 상태의 가치함수와 다음 상태의 가치함수 사이의 관계를 식으로 나타낸 것

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

  - 앞으로 받을 모든 보상을 한 번에 계산하는 것이 아니라, 값을 변수에 저장하고 루프를 돌며 참 값을 알아나감 → 다이내믹 프로그래밍과 관련
  - 좌변과 우변의 값이 다른 경우, 왼쪽의 현재 가치함수 값을 새로운 값으로 업데이트

- **계산 가능한 형태의 벨만 기대 방정식**

$$v_\pi(s) = \Sigma \pi(a|s)(r(s,a) + \gamma \Sigma P^a_{ss'} v_\pi(s'))$$

  - 기댓값에는 행동을 할 확률(정책 $\pi(a|s)$)과 행동을 했을 때 어떤 상태로 가게 되는 확률(상태 변환 확률 $P^a_{ss'}$)이 포함되어 있으므로 정책과 상태 변환 확률을 포함해서 계산

  그리드월드)

  - 상태 변환 확률이 1인 경우

$$v_\pi(s) = \Sigma \pi(a|s)(r(s,a) + \gamma v_\pi(s'))$$

    → (1) 각 행동에 대해 그 행동을 할 확률을 고려하고, (2) 각 행동을 했을 때 받을 보상과 (3) 다음 상태의 가치함수를 고려

벨만 최적 방정식

- **벨만 기대 방정식을 통한 가치함수 계산 과정**

$$v_{k+1}(s) \leftarrow \Sigma \pi(a|s)(r(s,a) + \gamma v_k(s'))$$

  - v_{k+1} : 현재 정책에 따라 k+1번째 계산한 가치함수
  - k+1번째 가치함수는 k번째 가치함수 중에서 주변 상태들 s'을 이용해 구하며, 모든 상태에 대해 동시에 계산 진행
  - 초깃값으로부터 반복적으로 계산하면 왼쪽 식과 오른쪽 식이 같아짐→ 현재 정책 π에 대한 **참 가치함수**를 구한 것

        - 참 가치함수 ≠ 최적 가치함수 : 참 가치함수는 "어떤 정책"을 따라서 움직였을 경우에 받게 되는 보상에 대한 참 기댓값
    - 단순히 현 정책에 대한 가치함수를 찾는 것이 아니라 더 좋은 정책으로 현재의 정책을 업데이트해나가야 함

    → 가치함수가 결국 정책이 얼마나 좋은지를 말해주는 것으로 모든 정책에 대해 가장 큰 가치함수를 주는 정책이 최적 정책

- **최적의 가치함수와 최적의 큐함수**

$$v^*(s) = max[v_\pi(s)]$$
$$q^*(s,a) = max[q_\pi(s,a)]$$

    - 최적 정책은 각 상태 s에서의 최적의 큐함수 중에서 가장 큰 큐함수를 가진 행동을 하는 것

    → 최적 정책은 최적 큐함수 q*만 안다면 구할 수 있음

$$\pi_*(\mathrm{s},\mathrm{a}) = \begin{cases} 1 & \text{if } \mathrm{a} = \mathrm{argmax}_{\mathrm{a}\in\mathrm{A}}\, \mathrm{q}_*(\mathrm{s},\mathrm{a}) \\ 0 & \text{otherwise} \end{cases}$$

- **벨만 최적 방정식**
    - 현재 상태의 가치함수가 최적이라는 것은 에이전트가 가장 좋은 행동을 선택한다는 것이며, 선택의 기준이 되는 것이 큐함수

$$v^*(s) = max[q^*(s,a)|S_t = s, A_t = a]$$

    - 큐함수를 가치함수로 고쳐서 표현한 것이 **벨만 최적 방정식(Bellman Optimality Equation)**

$$v^*(s) = maxE[R_{t+1} + \gamma v^*(S_{t+1})|S_t = s, A_t = a]$$

        - R_{t+1} 앞에 E가 있는 이유 : 다음 상태가 상태 변환 확률에 따라 달라지기 때문

- **큐함수에 대한 벨만 최적 방정식**

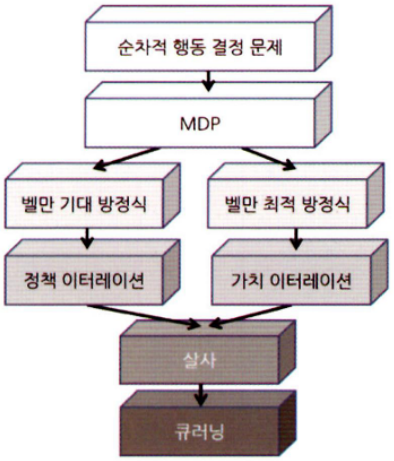
$$q^*(s,a) = E[R_{t+1} + \gamma\, max\, q^*(S_{t+1}, a')|S_t = s, A_t = a]$$

    - 최적 정책을 따라갈 때 현재 상태의 큐함수는 다음 상태에서 선택 가능한 행동 중에서 가장 높은 값의 큐함수를 1번 할인하고 보상을 더한 것과 같음

강화학습 기초2: 그리드월드와 다이내믹 프로그래밍

다이내믹 프로그래밍 1: 정책 이터레이션

강화학습 알고리즘의 흐름



순차적 행동 결정 문제를 MDP로 수학적으로 정의하고, 벨만 방정식을 통해 문제를 푼다. 벨만 방정식은 다이내믹 프로그래밍을 통해 풀 수 있다.

- 다이내믹 프로그래밍 두 가지 : **정책 이터레이션**과 **가치 이터레이션**
  - 정책 이터레이션과 가치 이터레이션은 이후에 살사(SARSA)와 큐러닝(Q-Learning)으로 이어짐

정책 이터레이션

다이내믹 프로그래밍의 한 종류로서 **벨만 기대 방정식**을 사용해 MDP로 정의되는 문제를 푼다.

- **목표** : 가장 높은 보상을 얻게 하는 정책을 찾는 것
- 처음에는 최적 정책을 알 수 없으므로, 무작위 정책으로부터 시작하여 계속 발전시켜 나감
- 정책 이터레이션 = **정책 평가(Policy Evaluation)** + **정책 발전(Policy Improvement)** 반복
  - 어떤 정책이 있을 때 그 정책을 정책 평가를 통해 얼마나 좋은지 평가하고, 그 평가를 기준으로 정책을 좀 더 나은 정책으로 발전시킴
  - 이러한 과정을 무한히 반복하면 정책은 최적 정책으로 수렴

정책 평가

정책이 얼마나 좋은지 판단하는 근거는 가치함수이다. 가치함수는 현재의 정책 π를 따라갔을 때 받을 보상에 대한 기댓값이므로, 현재 정책에 따라 받을 보상에 대한 정보가 정책의 가치가 된다.

- 정책 평가에서 사용하는 수식 = **벨만 기대 방정식**

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

  계산 가능한 합의 형태로 고치면,

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)(r_{(s,a)} + \gamma v_\pi(s'))$$

- 정책 평가는 π라는 정책에 대해 반복적으로 수행하는 것

$$v_{k+1}(s) = \sum_{a \in A} \pi(a \mid s)(r_{(s,a)} + \gamma v_k(s'))$$

- 계산 단계를 표현할 새로운 변수 k = 1, 2, 3, ... 를 설정
- k번째 가치함수를 통해 k+1번째 가치함수를 계산

한 번의 정책 평가 과정)

1. k번째 가치함수 행렬에서 현재 상태 s에서 갈 수 있는 다음 상태 s'에 저장돼 있는 가치함수 $v_k(s')$을 불러옴
2. $v_k(s')$에 할인율 γ를 곱하고 그 상태로 가는 행동에 대한 보상 R을 더함 →$r(s,a)+\gamma v_k(s')$
3. 2에서 구한 값에 그 행동을 할 확률(정책 값)을 곱함 → $\pi(a|s)(r(s,a)+\gamma v_k(s'))$
4. 3번을 모든 선택 가능한 행동에 대해 반복하고 그 값들을 더함
5. 4번 과정을 통해 더한 값을 k+1번째 가치함수 행렬의 상태 s자리에 저장
6. 1~5번 과정을 모든 s ∈ S에 대해 반복

→ 모든 상태에 대해 동시에 진행하며, 한 번의 정책평가로는 제대로 현재 정책에 대한 평가를 할 수 없으므로 v_1로 시작해서 무한히 반복하면 참 v_π가 됨

정책 발전

정책에 대한 평가를 마쳤다면 이를 바탕으로 정책을 발전시켜야 한다. 널리 알려진 방법으로 탐욕 정책 발전이 있다.

탐욕 정책 발전(Greedy Policy Improvement))

- **사용 함수** : 정책 평가를 통해 구한 것은 정책을 따랐을 때의 모든 상태에 대한 가치함수이므로 각 상태에 대해 어떤 행동을 하는 것이 좋은지를 알려면 **큐함수**를 사용

  $$q_\pi(s,a)=E_\pi[R_{t+1}+\gamma v_\pi(S_{t+1})\mid S_t=s,A_t=a]$$

  큐함수를 계산 가능한 형태로 고치면,

  $$q_\pi(s,a)=r_{(s,a)}+\gamma v_\pi(s')$$

  - s'은 행동 a를 통해 도달하게 되는 상태

- **탐욕 정책 발전** : 상태 s에서 선택 가능한 행동의 $q_\pi(s,a)$를 비교하고 가장 큰 큐함수를 가지는 행동을 선택함

  $$\pi'(s)=\mathrm{argmax}_{a\in A}\, q_\pi(s,a)$$

  - argmax : 가장 큰 큐함수를 가지는 **행동**을 반환하는 함수 (max와 달리 반환되는 것이 행동)
  - 탐욕 정책 발전을 통해 정책을 업데이트하면 이전 가치함수에 비해 업데이트된 정책으로 움직였을 때 받을 가치함수가 무조건 크거나 같음

다이내믹 프로그래밍 2: 가치 이터레이션

명시적인 정책과 내재적인 정책

- **정책 이터레이션** : 명시적인 정책이 있으며, 그 정책을 평가하는 도구로서 가치함수를 사용함. 정책과 가치함수가 명확히 분리되어 있음.
  - 정책이 독립적이므로 결정적인 정책(어떠한 상태에서 오직 하나의 행동만 선택하는)이 아니라 어떤 정책도 가능 → 대다수 정책은 확률적인 정책

    - 확률적인 정책을 고려해서 가치함수를 계산하려면 기댓값이 들어갈 수밖에 없음 → **벨만 기대 방정식** 사용
- **가치 이터레이션 :** 현재의 가치함수가 최적은 아니지만 최적이라고 가정하고, 그 가치함수에 대해 결정적인 형태의 정책을 적용. 정책이 명시적으로 표현되지 않고 가치함수 안에 내재적으로 포함
    - 가치함수를 업데이트하면 자동으로 정책도 발전 → 별도의 정책 발전 단계가 필요 없음
    - 정책값을 이용해 기댓값을 계산하는 부분 대신 **max**를 사용 → **벨만 최적 방정식** 사용

벨만 최적 방정식과 가치 이터레이션

다이내믹 프로그래밍은 큰 문제에 작은 문제들이 중첩된 경우에 큰 문제를 작은 문제로 쪼개고 작은 문제들의 결과를 서로 서로 이용해서 푸는 것이다.

- **벨만 기대 방정식**을 통해 전체 문제를 풀어서 나오는 답 = 현재 정책을 따라갔을 때 받을 참 보상

  → (1) 가치함수를 현재 정책에 대한 가치함수라고 가정하고 (2) 반복적으로 계산하면 (3) 결국 현재 정책에 대한 참 가치함수가 됨

$$\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

- **벨만 최적 방정식**을 통해 전체 문제를 풀어서 나오는 답 = 최적 가치함수

  → (1) 가치함수를 최적 정책에 대한 가치함수라고 가정하고 (2) 반복적으로 계산하면 (3) 결국 최적 정책에 대한 참 가치함수, 즉 최적 가치함수를 찾게 됨

$$v^*(s) = \max_a E[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]$$

    - 벨만 최적 방정식을 통해 문제를 푸는 가치 이터레이션에서는 따로 정책 발전이 필요없음
    - 시작부터 최적 정책을 가정했기 때문에 한 번의 정책 평가 과정을 거치면 최적 가치함수와 최적 정책이 구해지고 MDP가 풀림

벨만 최적 방정식 vs 벨만 기대 방정식)

- **벨만 기대 방정식** : 기댓값 E_π에 정책이 포함되기 때문에 정책을 고려
- **벨만 최적 방정식** : max를 취하기 때문에 정책을 고려할 필요가 없음

$$v_{k+1}(s) = \max_{a \in A}(r_{(s,a)} + \gamma v_k(s'))$$

    - 현재 상태에서 가능한 $R_{t+1} + \gamma v_k(S_{t+1})$값들 중에서 최고의 값으로 업데이트